Reviewer Pack — Minimal Local Index of Lie Groups and Circuit Monotone Width…

Entry b34e35ca03bb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 20:36:37 UTC

Minimal Local Index of Lie Groups and Circuit Monotone Width Inequality

Entry ID: `b34e35ca03bb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 20:36:37 UTC

1. Conjecture

Field A (mathematical branch): Lie Group Theory (Local Index) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every boolean circuit C with n inputs, the local index of its symmetry group G is upper-bounded by the monotone width w(C), i.e., |G| ≤ c^w(C) for some constant c.

Rationale (proposer's reasoning):

> The local index of a Lie group captures the complexity of the group's action on its Lie algebra, which can be related to the symmetries in circuit operations. This conjecture aims to connect this algebraic structure with the combinatorial properties of circuit monotone width, potentially revealing new insights into the symmetry-breaking processes in circuit computations.

Taxonomy category: `LieGroupLocalIndex_CircuitMonotoneWidth` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d42e4ef709834019`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson's correlation coefficient between |G| and c^w(C) across 30 randomly selected seeds is greater than 0.9.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"local index" AND Lie Groups` - `"circuit monotone width" AND Boolean circuits` - `"symmetry group" AND monotone width inequality`

Top relevant hits considered: - [http://arxiv.org/abs/1210.5855v4] A Lie-algebraic approach to the local index theorem on compact homogeneous spaces - [http://arxiv.org/abs/1401.0225v2] Local index theory for certain Fourier integral operators on Lie groupoids - [http://arxiv.org/abs/2311.07890v2] Index theorem for homogenous spaces of Lie groups - [http://arxiv.org/abs/2411.05766v1] A non-stabilizerness monotone from stabilizerness asymmetry - [http://arxiv.org/abs/1302.4157v3] On extremisers to a bilinear Strichartz inequality - [http://arxiv.org/abs/quant-ph/0305190v1] Symmetries of the Bell correlation inequalities - [s2:10.1007/978-3-030-58150-3_40] Succinct Monotone Circuit Certification: Planarity and Parameterized Complexity - [s2:10.4230/LIPIcs.STACS.2019.41] Lower Bounds for DeMorgan Circuits of Bounded Negation Width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        # Generate a random boolean circuit with n inputs
        if n == 1:
            return ['0'] * (2**n)
        else:
            subcircuits = [generate_circuit(n-1) for _ in range(3)]
            return [f'({sub[0]} & {sub[1]}) | {sub[2]}' for sub in zip(*subcircuits)]

    def local_index(circuit):
        # Placeholder function to compute the local index
        # This is a dummy implementation and should be replaced with actual computation
        return len(set(circuit))

    def monotone_width(circuit):
        # Placeholder function to compute the monotone width
        # This is a dummy implementation and should be replaced with actual computation
        return 1

    n = random.randint(5, 40)
    circuit = generate_circuit(n)
```

```
    li = local_index(circuit)
    mw = monotone_width(circuit)

    return {
        "metric_name": "Local Index vs Monotone Width",
        "metric_value": li,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the Pearson's correlation coefficient could not be calculated to determine if the conjecture is supported. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14112
2	propose	ollama_remote	glm4:latest	0	0	12501
3	preregistration	ollama_remote	glm4:latest	0	0	9007
4	novelty	ollama_remote	glm4:latest	0	0	7934
5	novelty	ollama_remote	glm4:latest	0	0	10267
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15369
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9243
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7190
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7411
10	judge	ollama_remote	glm4:latest	0	0	36751

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 129785 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b34e35ca03bb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b34e35ca03bb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b34e35ca03bb.tar.gz` (if generated)